DUOC UC - ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Especificación de Requisitos de Software

Proyecto: Sitio web Clínica Nutricional NutriVida

Revisión*: 2.0*

17/10/2026

Especificación de Requisitos según estándar de IEEE 830.

-

Contenido

Ficha del documento **4**

1. Introducción **5**

1.1. Propósito 5

1.2. Ámbito del Sistema 6

1.3. Definiciones, Acrónimos y Abreviaturas 7

1.4. Referencias 8

1.5. Visión General del Documento 8

2. Descripción General **9**

2.1. Perspectiva del Producto 9

2.2. Funciones del Producto 9

2.3. Características de los Usuarios 10

Usuario visitante 10

Paciente 11

Administrador 11

2.4. Restricciones 11

2.5. Suposiciones y Dependencias 11

2.6. Requisitos Futuros 12

3. Requisitos Específicos **13**

3.1 Requisitos comunes de las interfaces 13

3.1.1 Interfaces de usuario *13*

3.1.2 Interfaces de hardware *14*

3.1.3 Interfaces de software *14*

3.1.4 Interfaces de comunicación *14*

3.2 Requisitos funcionales 15

3.2.1 RF-01 Visualizar página principal 15

3.2.2 RF-02 Navegar por el sitio público 15

3.2.3 RF-03 Visualizar información institucional 16

3.2.4 RF-04 Visualizar nutricionistas 16

3.2.5 RF-05 Consultar detalle del nutricionista 16

3.2.6 RF-06 Visualizar publicaciones del blog 17

3.2.7 RF-07 Consultar detalle del blog 17

3.2.8 RF-08 Registrar una cuenta de paciente 17

3.2.9 RF-09 Validar datos del registro 18

3.2.10 RF-10 Validar RUN del paciente 18

3.2.11 RF-11 Seleccionar región y comuna 18

3.2.12 RF-12 Iniciar sesión 19

3.2.13 RF-13 Validar inicio de sesión 19

3.2.14 RF-14 Procesar formulario de contacto 19

3.2.15 RF-15 Validar formulario de contacto 20

3.2.16 RF-16 Mostrar validaciones y retroalimentación 20
3.2.17 RF-17 Seleccionar nutricionista para una cita 20
3.2.18 RF-18 Seleccionar fecha y hora 21
3.2.19 RF-19 Registrar motivo de consulta 21
3.2.20 RF-20 Validar fecha y conflictos 21
3.2.21 RF-21 Agregar consulta al carrito 22
3.2.22 RF-22 Visualizar resumen del carrito 22
3.2.23 RF-23 Calcular total del carrito 22
3.2.24 RF-24 Eliminar consulta del carrito 23
3.2.25 RF-25 Confirmar agendamiento 23
3.2.26 RF-26 Conservar citas y carrito 23
3.2.27 RF-27 Acceder al panel administrador 24
3.2.28 RF-28 Gestionar pacientes 24
3.2.29 RF-29 Validar datos de pacientes 24
3.2.30 RF-30 Gestionar nutricionistas 25
3.2.31 RF-31 Aplicar los precios de los planes 25
3.2.32 RF-32 Visualizar y buscar datos administrativos 25
3.2.33 RF-33 Filtrar el catálogo por categoría 26
3.2.34 RF-34 Consultar ofertas y beneficios 26
3.2.35 RF-35 Preseleccionar un plan desde un enlace 26
3.2.36 RF-36 Revisar los datos antes de confirmar 27
3.2.37 RF-37 Simular un fallo de confirmación 27
3.2.38 RF-38 Cerrar sesión y adaptar el menú 27
3.2.39 RF-39 Conservar integridad al eliminar registros 28
3.2.40 RF-40 Resolver rutas y desplazamiento 28
3.3 Requisitos no funcionales 28
3.3.1 Requisitos de rendimiento *28*
3.3.2 Seguridad *29*
3.3.3 Fiabilidad *29*
3.3.4 Disponibilidad *30*
3.3.5 Mantenibilidad *30*
3.3.6 Portabilidad *30*
3.3.7 Usabilidad y accesibilidad 31
3.4 Otros Requisitos 31
3.4.1 Reglas de validación 31
3.4.2 RNF-09 Proceso de pruebas y cobertura 32
3.4.3 RNF-10 Publicación en AWS EC2 33
3.4.4 Trazabilidad y criterios de entrega 33

-

Ficha del documento

Fecha	Revisión	Autor	Modificación
17/10/2026	*2.0*	*Nitsi Sepúlveda, Jonathan Rivera y Sofía González.*	*Actualización a React de la EV2: RF-01 a RF-40 y RNF-01 a RNF-10, catálogo, ofertas, revisión de reservas, CRUD, validaciones, Vitest, cobertura y publicación en AWS EC2.*
12/09/2026	*1.0*	*Nitsi Sepúlveda, Jonathan Rivera y Sofía González.*	*Primera versión de la propuesta, definición del alcance, usuarios, herramientas y requisitos del sistema NutriVida.*

Documento validado por las partes en fecha:

Por el cliente

[Firma]

Sr./Sra.

Por la empresa suministradora

[Firma]

Sr./Sra.

-

1. Introducción

Este documento presenta la segunda versión de la Especificación de Requisitos de Software de la Clínica Nutricional NutriVida, correspondiente a la entrega del 17 de octubre de 2026. Describe el alcance final del frontend React, sus usuarios, interfaces, reglas de negocio, requisitos y criterios de verificación.

La versión 2.0 actualiza la propuesta de la EV1 para la aplicación construida con React y Vite. Incorpora componentes reutilizables, propiedades, estado, contextos, navegación con React Router, formularios validados, persistencia local, pruebas con Vitest y publicación de la compilación en AWS EC2 mediante Nginx.

El sistema permite conocer la clínica y sus profesionales, consultar planes por categoría y ofertas, leer artículos, registrar pacientes, iniciar y cerrar sesión, utilizar contacto, agendar consultas, revisar datos antes de confirmar reservas y administrar pacientes y nutricionistas.

Los requisitos describen comportamientos observables del alcance frontend. Su verificación se apoya en pruebas automatizadas, revisión responsive y evidencias de publicación. No se incluye backend ni procesamiento real de pagos o correos.

La especificación constituye una base común para el equipo y el evaluador: permite contrastar la implementación con comportamientos esperados y reconocer las condiciones que limitan su uso. La descripción distingue la navegación informativa, las operaciones que modifican registros y los procesos que dependen del almacenamiento del navegador. Esta distinción facilita interpretar los resultados de las pruebas y evita atribuir a la aplicación servicios que pertenecen a una infraestructura futura.

La versión documentada corresponde a la evolución del proyecto académico desde páginas HTML hacia una aplicación React. La continuidad entre etapas se mantiene mediante los identificadores de requisitos, mientras que las reglas actualizadas reflejan las decisiones adoptadas durante la implementación, como la tarifa por plan, la revisión previa de las reservas y la separación entre el sitio público y la administración.

1.1. Propósito

El propósito de este documento es definir los requisitos funcionales y no funcionales de NutriVida para la Evaluación Parcial 2 de DSY1104 Desarrollo Full Stack II.

La ERS orienta el diseño, implementación y evaluación de la aplicación React. Define responsabilidades de los componentes, reglas de validación, gestión y persistencia de datos y criterios de aceptación. Se conserva la continuidad con RF-01 a RF-32 y RNF-01 a RNF-08 y se incorporan RF-33 a RF-40 y RNF-09 a RNF-10.

El documento está dirigido al docente de la asignatura Desarrollo Full Stack II, Hernán Saavedra, a los integrantes del equipo de desarrollo y a las personas que participen en la revisión del proyecto.

-

Para el equipo de desarrollo, la ERS permite revisar que los formularios, contextos y páginas compartan las mismas reglas. Para el docente, proporciona criterios observables con los cuales relacionar la demostración, el código fuente y la evidencia de pruebas. Para un eventual mantenedor, identifica las dependencias y las limitaciones que deberá considerar antes de modificar o ampliar el producto.

Los términos deberá y debe expresan condiciones exigidas dentro de esta versión. Los criterios de aceptación describen resultados verificables, pero no sustituyen los casos de prueba detallados ni representan una certificación de calidad. Cuando una capacidad se identifica como futura, su descripción establece una posible evolución y no un compromiso de implementación para esta entrega.

1.2. Ámbito del Sistema

El sistema NutriVida corresponde a una aplicación web frontend para una clínica nutricional. La versión de esta entrega se encuentra en fase2/proyecto_etapa2_react/ y la versión HTML anterior se conserva en fase1/proyecto_etapa1_html/.

Los visitantes pueden conocer la clínica, consultar nutricionistas, planes, categorías, ofertas y artículos, utilizar el formulario de contacto, registrarse, iniciar sesión y completar el flujo de reservas con datos de contacto.

Cada nutricionista presenta fotografía, nombre, especialidad, experiencia y modalidad. El precio pertenece al plan elegido: consulta inicial $35.000 CLP, plan integral $60.000 CLP y nutrición deportiva $50.000 CLP. Elegir otro profesional no agrega un cobro adicional.

El panel administrativo dispone de dashboard y gestión de pacientes y nutricionistas mediante creación, listado, búsqueda, edición y eliminación. Su interfaz se habilita para el rol administrador de la sesión simulada.

En la EV2 el sistema funciona como prototipo frontend React. Los módulos JavaScript aportan datos iniciales y localStorage conserva pacientes, profesionales, sesión, carrito y citas en el navegador utilizado.

No se procesan pagos reales, no existe base de datos remota y no se envían correos efectivos. Las credenciales locales y el control de rutas por rol son una simulación frontend, sin autenticación segura conectada a un servidor.

El flujo principal comienza en la consulta del catálogo y termina en una confirmación local de citas. El carrito representa una selección temporal de atenciones; su confirmación registra reservas de demostración y muestra sus identificadores. El uso de términos comerciales, como precio y total, permite representar el costo del servicio, sin que exista un cobro bancario ni la emisión de un documento tributario.

-

El alcance administrativo se concentra en mantener los registros necesarios para la demostración. Las altas, modificaciones y eliminaciones se reflejan en el almacenamiento del mismo navegador y origen. El sistema no administra diagnósticos, tratamientos, expedientes clínicos ni antecedentes de salud reales; estos procesos requieren un diseño de seguridad y persistencia distinto al contemplado en la EV2.

Los principales objetivos son:

- Entregar información clara sobre la Clínica NutriVida.
- Facilitar la consulta de servicios por categoría, ofertas y profesionales.
- Permitir agendar consultas futuras sin conflictos de profesional, fecha y horario.
- Revisar las consultas y datos del paciente antes de confirmar la reserva simulada.
- Facilitar la gestión de pacientes y nutricionistas.
- Aplicar React, Vite, JSX, React Router, React Bootstrap, CSS y JavaScript.
- Verificar el sistema con Vitest y cobertura, publicarlo en AWS y gestionar los cambios mediante GitHub.

1.3. Definiciones, Acrónimos y Abreviaturas

ERS: Especificación de Requerimientos de Software.

React y JSX: biblioteca de interfaz y sintaxis utilizadas para construir componentes reutilizables.

CSS: lenguaje de estilos; App.css, index.css y styles/Admin.css organizan la presentación de la aplicación.

JavaScript: lenguaje de la lógica, interacción, validaciones y módulos de datos.

Bootstrap y React Bootstrap: grillas, utilidades y componentes de interfaz para adaptar el contenido a distintos tamaños.

CRUD: Operaciones de crear, leer o listar, actualizar y eliminar información.

RUN: Rol Único Nacional utilizado para identificar a una persona en Chile.

Usuario visitante: Persona que navega por el sitio sin iniciar sesión.

Paciente: cuenta registrada con rol paciente y sesión local; sus datos pueden precargarse en la revisión de la reserva.

Administrador: Usuario encargado de gestionar pacientes y nutricionistas.

Carrito de reservas: conjunto temporal de consultas y sus precios, con revisión y confirmación simulada.

localStorage y JSON: persistencia en el navegador, vinculada al origen del sitio y al dispositivo utilizado.

-

Frontend: Parte visual e interactiva de una aplicación web que se ejecuta en el navegador.

Responsive: Capacidad de una página para adaptarse a distintos tamaños de pantalla.

Vite y React Router: entorno de desarrollo y compilación, y biblioteca de rutas, layouts y parámetros de navegación.

Git y GitHub: control de versiones, ramas y revisión de cambios mediante Pull Requests.

Props, estado y Context: datos recibidos por componentes, datos que actualizan la interfaz y mecanismo para compartir sesión, registros y reservas.

Vitest y Testing Library: herramientas para ejecutar pruebas y verificar el DOM y las interacciones mediante casos de uso y mocks.

Cobertura v8: medición de sentencias, ramas, funciones y líneas ejecutadas durante las pruebas.

AWS EC2 y Nginx: infraestructura de publicación y servidor web que entrega la compilación estática.

1.4. Referencias

Para elaborar esta ERS se utilizaron las siguientes referencias:

- Documento del caso Clínica Nutricional NutriVida.
- Catálogo de la Clínica NutriVida.
- Instrucciones y rúbrica de la Evaluación Parcial 2 de DSY1104.
- Guía docente de pruebas unitarias con React, Vite y Vitest.
- Guías docentes 2.1.2 y 2.2.2 para React y AWS EC2, y documentación de la primera entrega.
- Plantilla ERS basada en el estándar IEEE 830.
- Repositorio público del proyecto en GitHub: https://github.com/soofigonb/nutrivida-tienda-online
- Documentación oficial de React, Vite, React Router, React Bootstrap, React Hook Form y Vitest, e informe de pruebas del proyecto.

1.5. Visión General del Documento

Este documento se organiza en tres secciones principales.

La primera sección presenta el propósito, alcance, conceptos y referencias del proyecto. La segunda sección describe la perspectiva del producto, sus funciones, tipos de usuarios, restricciones, dependencias y posibles mejoras futuras.

La tercera sección define interfaces, requisitos funcionales y no funcionales, validaciones, pruebas y despliegue de la versión 2.0.

-

2. Descripción General

Esta sección presenta el contexto general del sistema NutriVida. Se describe su relación con otras herramientas, las funcionalidades principales, los usuarios, las restricciones técnicas, las dependencias y las funcionalidades que podrían incorporarse en futuras versiones.

2.1. Perspectiva del Producto

NutriVida es una aplicación frontend que se ejecuta en el navegador y se publica como una compilación estática servida con Nginx.

La aplicación se organiza en components, pages, context, data, utils y styles. App.jsx define rutas y layouts; main.jsx monta React. AuthContext comparte la sesión, DatosContext gestiona registros y CarritoContext controla las reservas.

La composición mediante componentes permite reutilizar navegación, tarjetas, formularios y listados sin duplicar su estructura. Las páginas coordinan el contenido asociado a cada ruta y las utilidades concentran operaciones comunes, como validación, formato monetario y acceso al almacenamiento. Esta separación facilita localizar los cambios y comprobar sus efectos en los módulos que los consumen.

AuthContext mantiene la información de sesión y expone las operaciones de registro, ingreso y cierre. DatosContext concentra la información administrativa y comunica los cambios a sus consumidores. CarritoContext administra las consultas pendientes y la confirmación de citas. Los contextos comparten datos que deben ser coherentes entre páginas; los estados de presentación, como abrir un detalle o expandir un menú, se mantienen en los componentes responsables.

Las propiedades permiten comunicar datos y configuraciones a los componentes reutilizables. El estado controla la actualización de la interfaz después de una interacción. La persistencia complementa ambos mecanismos: guarda una representación JSON para recuperarla al volver a abrir o recargar la aplicación, sin convertir el navegador en un servicio de base de datos compartido.

La EV2 no incorpora un backend ni una base de datos remota. Los datos iniciales se encuentran en módulos JavaScript y la persistencia local utiliza JSON en localStorage. Las actualizaciones de los contextos se reflejan en la interfaz.

Bootstrap y React Bootstrap apoyan el diseño responsive. React Hook Form gestiona formularios. Vitest, Testing Library y jsdom verifican el comportamiento. GitHub conserva el código y Nginx publica dist en una instancia EC2 del laboratorio AWS Academy.

2.2. Funciones del Producto

Las funciones generales implementadas para la versión 2.0 son las siguientes:

-

- Mostrar la clínica, historia, misión, equipo y video educativo con su fuente.
- Navegar entre rutas públicas y secciones de inicio y mostrar una página no encontrada.
- Mostrar dinámicamente a los nutricionistas.
- Mostrar fotografía, especialidad, experiencia y modalidad de los profesionales.
- Mostrar publicaciones y detalles de consejos de nutrición.
- Registrar datos de pacientes.
- Validar el RUN, correo y contraseñas.
- Actualizar las comunas según la región seleccionada.
- Permitir inicio y cierre de sesión y adaptar el menú al rol.
- Validar el contacto y mostrar su resultado de envío simulado, sin generar un correo real.
- Mostrar mensajes de validación y resultados según la interacción y el envío de cada formulario.
- Seleccionar un nutricionista para una consulta.
- Seleccionar la fecha, hora y motivo de la consulta.
- Impedir fechas y horas pasadas y reservas repetidas del mismo profesional.
- Agregar una consulta a un carrito simulado.
- Mostrar los precios de los planes y recalcular el total de las consultas.
- Eliminar consultas del carrito.
- Revisar nombre y correo del paciente y confirmar las citas con un resumen e identificadores.
- Guardar información en localStorage.
- Administrar pacientes.
- Administrar nutricionistas.
- Aplicar precios por plan y presentar categorías y ofertas con sus condiciones.

Revisar el resumen de la reserva y volver al carrito antes de confirmar.

Conservar datos ante errores y evitar duplicados al reintentar.

Filtrar servicios por categoría y consultar las condiciones del beneficio de primera consulta.

2.3. Características de los Usuarios

Usuario visitante

Una persona sin sesión puede consultar el sitio, filtrar servicios, revisar beneficios, usar contacto, registrarse o iniciar sesión. También puede agendar y confirmar consultas ingresando nombre y correo en la revisión del carrito.

No necesita conocimientos técnicos avanzados. Solo debe manejar funciones básicas de navegación web.

-

Paciente

El paciente puede iniciar sesión, consultar los servicios y reservar atenciones. El nombre y correo se precargan en el carrito y pueden editarse para la cita sin cambiar la cuenta. No tiene acceso al panel administrativo.

Debe poseer conocimientos básicos de navegación web y formularios.

Administrador

El administrador accede al dashboard, búsquedas y CRUD de pacientes y nutricionistas. Debe validar los formularios, confirmar eliminaciones y respetar los bloqueos por citas o reservas asociadas.

Debe poseer conocimientos básicos sobre navegación web, formularios y administración de información.

2.4. Restricciones

El desarrollo del proyecto estará sujeto a las siguientes restricciones:

- La interfaz deberá implementarse mediante componentes React y JSX con estructura semántica.
- Los estilos personalizados deberán encontrarse en archivos CSS externos.
- Las interacciones deberán utilizar JavaScript, propiedades y estado de React, y contextos para datos compartidos.
- La lógica deberá distribuirse por responsabilidad entre páginas, componentes, contextos y utilidades; no se concentrará en app.js.
- Se utilizarán Bootstrap, React Bootstrap y media queries para el diseño responsive.
- El sistema deberá ser compatible con computadores, tabletas y teléfonos.
- El proyecto deberá ejecutarse en un navegador web moderno.
- Los datos deberán persistirse como JSON en localStorage, con tratamiento de fallos de lectura y escritura.
- El proyecto no utilizará un backend durante esta evaluación.
- El proyecto no utilizará una base de datos remota.
- El carrito no procesará pagos reales.
- El sistema no enviará correos electrónicos reales.
- Cada integrante deberá trabajar mediante ramas y commits identificables en GitHub.
- Los mensajes de los commits deberán describir claramente los cambios realizados.

2.5. Suposiciones y Dependencias

Para el funcionamiento del sistema se consideran las siguientes suposiciones y dependencias:

- El usuario tendrá acceso a un navegador actualizado.

- El navegador tendrá JavaScript habilitado.
- El dispositivo permitirá utilizar localStorage.
- El usuario tendrá conexión a Internet para acceder a AWS y cargar recursos externos como fuentes y video. Bootstrap se incorpora como dependencia del proyecto.
- Las imágenes y archivos del proyecto estarán disponibles en las rutas correspondientes.
- Los datos se conservan en el navegador y origen utilizados. No se trasladan automáticamente entre localhost, IP públicas distintas, navegadores o dispositivos.
- El usuario completará los datos solicitados en los formularios.
- El repositorio de GitHub estará disponible para los integrantes del equipo.
- Los integrantes coordinarán la integración de sus cambios para evitar conflictos.
- Los precios de los planes son valores de demostración del catálogo. La comprobación de fechas usa la hora del dispositivo.

La publicación requiere una instancia EC2 activa dentro de AWS Academy y presupuesto disponible.

El desarrollo y las pruebas requieren Node.js 24, npm y las dependencias registradas en package-lock.json.

La instalación de dependencias debe respetar package-lock.json para reducir diferencias entre el entorno local y el servidor. La compilación de Vite produce recursos estáticos que Nginx puede entregar sin mantener abierto el servidor de desarrollo. La ejecución de npm run dev corresponde al trabajo local y no constituye el mecanismo de publicación utilizado en AWS.

La continuidad de los datos depende del origen desde el cual se abre la aplicación. Un cambio de IP pública puede mostrar un almacenamiento inicialmente vacío porque el navegador lo considera un origen diferente. Esta situación no debe interpretarse como una migración o sincronización automática de registros. Asimismo, eliminar los datos del sitio en el navegador elimina la persistencia disponible para ese origen.

2.6. Requisitos Futuros

Las siguientes funciones quedan fuera del alcance de la EV2 y podrían incorporarse en versiones posteriores:

- Implementación de un backend.
- Conexión con una base de datos.
- Registro y autenticación real de usuarios.
- Recuperación de contraseñas.
- Envío real de correos electrónicos.
- Integración con una pasarela de pago.
- Confirmación de citas mediante correo electrónico.
- Gestión avanzada de disponibilidad, duración y agendas particulares de cada profesional.
- Historial de consultas de los pacientes.
- Ficha clínica del paciente.
- Permisos verificados por un backend, con autenticación segura y control de acceso real.

- Implementación de medidas avanzadas de seguridad.
- Publicación con dominio propio y HTTPS; la versión actual utiliza HTTP en AWS EC2.
- Generación de reportes administrativos.

3. Requisitos Específicos

Los requisitos específicos describen el comportamiento esperado de la versión React de NutriVida. Cada requisito funcional incluye sus actores, descripción y criterio de aceptación. Los precios por nutricionista de la propuesta anterior se sustituyen por precios de los planes.

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

El sitio público comparte Navbar y Footer mediante un layout con Outlet. Presenta inicio, servicios, ofertas, registro, login, agendamiento, carrito y artículos. Los enlaces por sección utilizan el hash de la URL.

El panel administrativo utiliza AdminLayout con menú lateral en escritorio y distribución adaptada en pantallas pequeñas, dashboard, listados y formularios propios.

Los formularios incluyen etiquetas asociadas, ayudas y errores por campo. El estado controla la promoción, detalle de profesionales, filtros, navegación móvil y pasos del carrito. Se mantiene una presentación coherente entre páginas.

La interfaz se adapta a 360, 768 y 1280 px y anchos intermedios mediante grillas, componentes Bootstrap y media queries. Las tarjetas, tablas, formularios, video y menús deben conservar su contenido y funcionalidad.

En pantallas pequeñas, la navegación pública debe poder desplegarse mediante su control de menú y permitir seleccionar cada destino. En formularios, las etiquetas y los mensajes deben permanecer junto a los campos correspondientes. Los listados administrativos deben permitir consultar sus datos sin superponer las acciones de edición y eliminación. La adaptación se verifica sobre el contenido y las operaciones, además de la apariencia visual.

El layout público y el administrativo responden a objetivos distintos: el primero facilita explorar servicios y reservar, mientras el segundo prioriza la gestión de registros. La separación de layouts evita presentar el menú administrativo como parte de la navegación de un visitante y permite mantener una organización consistente dentro de cada contexto de uso.

-

3.1.2 Interfaces de hardware

El sistema no requiere hardware especializado. Se podrá utilizar desde computadores, tabletas y teléfonos móviles que cuenten con un navegador web moderno.

El usuario podrá interactuar mediante teclado, mouse o pantalla táctil, dependiendo del dispositivo utilizado.

3.1.3 Interfaces de software

La aplicación utiliza React, Vite, React Router, React Bootstrap, Bootstrap Icons, React Hook Form y CSS. El desarrollo requiere Node.js 24, npm y Git. Vitest, Testing Library, user-event, jest-dom, jsdom y coverage-v8 configuran las pruebas. Las versiones se registran en package-lock.json.

Herramienta	Propósito
React y JSX	Construir componentes reutilizables con propiedades y estado.
Vite	Ejecutar el desarrollo y generar la compilación de producción.
React Router	Gestionar rutas, layouts y parámetros de navegación.
Bootstrap y React Bootstrap	Adaptar grillas, formularios y componentes a distintos tamaños.
React Hook Form	Gestionar formularios y validaciones de datos.
localStorage y JSON	Conservar registros, sesión, carrito y citas en el navegador.
Vitest y Testing Library	Verificar lógica, DOM e interacciones y generar cobertura con v8.
Node.js, npm, Git y GitHub	Instalar dependencias y gestionar el desarrollo y sus versiones.
AWS EC2 y Nginx	Servir la compilación y resolver la recarga de rutas públicas.

3.1.4 Interfaces de comunicación

El servidor Nginx entrega los archivos compilados por HTTP en el puerto 80. React Router resuelve las rutas y try_files permite recargarlas sin errores 404 del servidor.

SSH por el puerto 22 permite administrar EC2 desde la IP autorizada. Los recursos externos, incluidos el video y las fuentes, requieren conexión a sus proveedores. El formulario de contacto y el carrito no se comunican con pasarelas de pago o servicios de correo reales.

La información funcional se procesa en el navegador. Se utilizan las claves pacientesNutriVida, nutricionistasNutriVida, sesionNutriVida, carritoNutriVida y citasNutriVida para la persistencia local.

La comunicación HTTP entrega el documento principal y los recursos compilados. Una vez cargada la aplicación, los cambios de ruta y las operaciones sobre registros se resuelven en el frontend. No existe una API propia que valide credenciales, procese reservas o sincronice pacientes entre equipos. El acceso SSH se utiliza únicamente para administrar el servidor y no forma parte del flujo de los usuarios del sitio.

3.2 Requisitos funcionales

Los requisitos funcionales incluyen información pública, navegación, registro, sesión, contacto, servicios, agendamiento, revisión y confirmación de reservas y administración de datos. Su prioridad es alta dentro del alcance de la EV2.

Se conservan los identificadores RF-01 a RF-32 con contenido actualizado y se incorporan RF-33 a RF-40 para las funciones añadidas en esta etapa.

3.2.1 RF-01 Visualizar página principal

Actores: visitante y paciente.
Descripción: Mostrar la presentación de la clínica, imagen de portada y accesos a profesionales y agendamiento.
Criterio de aceptación: Al abrir / se visualiza el contenido principal con Navbar y Footer.

Condiciones de uso: La aplicación debe estar cargada en el navegador. La página de inicio constituye el punto de entrada para quien desea conocer la clínica y acceder a sus servicios.

Desarrollo y resultado esperado: El usuario revisa la portada y utiliza los accesos disponibles para continuar hacia otras secciones. El resultado esperado es una presentación reconocible y una navegación consistente con el resto del sitio público; visitar el inicio no modifica pacientes, citas ni consultas del carrito.

3.2.2 RF-02 Navegar por el sitio público

Actores: todos los usuarios.
Descripción: Permitir acceder a Home, Nosotros, Servicios, Ofertas, Blogs, Agendar, Contacto, sesión y carrito.
Criterio de aceptación: Los enlaces abren su ruta o sección y el menú móvil se cierra al seleccionar.

Condiciones de uso: La persona puede navegar con o sin una sesión activa. Las opciones de cuenta se presentan de acuerdo con el estado de sesión y el rol reconocido por el frontend.

Desarrollo y resultado esperado: Cada enlace debe conducir al destino que anuncia su texto. Los enlaces a secciones de inicio deben conservar su referencia en la URL. En el menú desplegable, seleccionar una opción debe cerrar la expansión y permitir que el usuario continúe consultando el contenido elegido.

-

3.2.3 RF-03 Visualizar información institucional

Actores: visitante y paciente.
Descripción: Mostrar historia, misión y equipo, con un video educativo identificado como recurso externo.
Criterio de aceptación: La sección Nosotros presenta el contenido y un reproductor de video accesible.

Condiciones de uso: El contenido institucional debe encontrarse disponible en la sección Nosotros. La reproducción del recurso audiovisual depende de la conexión y del proveedor externo.

Desarrollo y resultado esperado: La información textual permite comprender la presentación de la clínica aunque el usuario no reproduzca el video. El recurso audiovisual se incorpora como apoyo educativo y su fuente debe ser identificable; no se presenta como una grabación clínica ni como una producción propia del equipo.

3.2.4 RF-04 Visualizar nutricionistas

Actores: visitante y paciente.
Descripción: Renderizar los profesionales guardados con fotografía, nombre y especialidad.
Criterio de aceptación: Las tarjetas reflejan los datos administrativos y no añaden un precio individual.

Condiciones de uso: Debe existir una fuente válida de profesionales, obtenida de los datos iniciales o de los registros locales gestionados por administración.

Desarrollo y resultado esperado: El listado debe construir las tarjetas a partir de esos registros para mantener coherencia entre la gestión administrativa y la información pública. La verificación comprueba correspondencia entre profesional, fotografía y especialidad. La ficha no incorpora una tarifa individual, porque el costo de la reserva se determina por el plan.

3.2.5 RF-05 Consultar detalle del nutricionista

Actores: visitante y paciente.
Descripción: Abrir información ampliada del profesional seleccionado y permitir cerrar o cambiar el detalle.
Criterio de aceptación: La selección muestra experiencia y modalidad del profesional correspondiente.

Condiciones de uso: El usuario debe seleccionar un profesional del listado. La información ampliada debe corresponder al registro seleccionado y conservar su identidad.

Desarrollo y resultado esperado: Al abrir el detalle, se muestran los antecedentes disponibles de experiencia y modalidad. Cerrar la vista permite continuar navegando y seleccionar otra persona. Cambiar el detalle no debe alterar los datos del catálogo ni agregar automáticamente una consulta al carrito.

-

3.2.6 RF-06 Visualizar publicaciones del blog

Actores: visitante y paciente.
Descripción: Mostrar al menos dos artículos con imagen, título y acceso a su lectura.
Criterio de aceptación: El inicio presenta ambos artículos y enlaces diferenciados.

Condiciones de uso: Los artículos deben estar definidos en la fuente de contenido de la aplicación y contar con sus enlaces de navegación.

Desarrollo y resultado esperado: El visitante revisa el resumen de cada publicación y elige el artículo que desea consultar. El listado debe permitir reconocer los contenidos disponibles y conducir al detalle correspondiente, manteniendo los elementos compartidos del sitio. La lectura no exige registro ni modifica datos de la sesión.

3.2.7 RF-07 Consultar detalle del blog

Actores: visitante y paciente.
Descripción: Abrir el artículo completo correspondiente al enlace seleccionado.
Criterio de aceptación: Cada ruta de blog muestra su título y contenido propio con el layout público.

Condiciones de uso: La ruta del artículo debe corresponder a una publicación definida en la aplicación.

Desarrollo y resultado esperado: Al acceder desde el listado o mediante la dirección interna, se muestra el contenido del artículo seleccionado con su presentación pública. La navegación permite continuar hacia el sitio. El contenido tiene un propósito informativo y no constituye una consulta personalizada ni genera una reserva por su lectura.

3.2.8 RF-08 Registrar una cuenta de paciente

Actores: visitante.
Descripción: Crear el registro local con datos personales, contacto, dirección y contraseña.
Criterio de aceptación: Un envío válido conserva los registros anteriores, asigna rol paciente y navega a /login.

Condiciones de uso: La persona debe completar el formulario de registro con los datos requeridos. El almacenamiento de pacientes debe poder leerse y admitir la escritura de un nuevo registro.

Desarrollo y resultado esperado: Antes de persistir, se aplican normalización y validaciones. Si la operación concluye, el paciente nuevo se incorpora sin reemplazar los anteriores y se dirige al inicio de sesión. El formulario público asigna el rol paciente, aunque se intente proporcionar otro rol desde los datos enviados.

-

3.2.9 RF-09 Validar datos del registro

Actores: visitante.
Descripción: Comprobar obligatoriedad, formatos y duplicidad de RUN y correo antes de guardar.
Criterio de aceptación: Los errores se muestran en el campo correspondiente y el formulario no crea un paciente.

Condiciones de uso: El formulario debe disponer de los datos introducidos y de los registros necesarios para comprobar duplicidad.

Desarrollo y resultado esperado: La validación considera campos obligatorios, formatos permitidos y reglas específicas. La existencia de errores impide registrar y permite corregir la entrada. Si el contexto devuelve un RUN o correo duplicado, el mensaje debe asociarse al campo afectado; un fallo general de almacenamiento debe informar que no se completó la operación.

3.2.10 RF-10 Validar RUN del paciente

Actores: visitante y administrador.
Descripción: Normalizar el RUN y verificar su estructura y dígito verificador.
Criterio de aceptación: Un RUN con dígito incorrecto se rechaza; variantes de formato no evitan detectar duplicados.

Condiciones de uso: El RUN se recibe como texto y puede contener los separadores habituales de su representación.

Desarrollo y resultado esperado: La normalización permite comparar variantes que representan el mismo identificador. La validación del dígito verificador se aplica antes de guardar el registro. Un formato visual distinto no debe permitir crear dos pacientes con el mismo RUN; esta comprobación no equivale a consultar un registro civil externo.

3.2.11 RF-11 Seleccionar región y comuna

Actores: visitante y administrador.
Descripción: Actualizar las comunas según la región; deshabilitar la comuna hasta seleccionar región.
Criterio de aceptación: Cambiar de región limpia la comuna previa y muestra solo sus opciones disponibles.

Condiciones de uso: El formulario debe utilizar el catálogo de regiones y comunas incorporado al proyecto.

Desarrollo y resultado esperado: El usuario selecciona primero una región y luego una de sus comunas disponibles. Al cambiar la región, se elimina la selección anterior para evitar conservar una combinación incongruente. La comprobación verifica tanto la habilitación del campo como el contenido de sus opciones y la limpieza de la comuna previa.

-

3.2.12 RF-12 Iniciar sesión

Actores: paciente y administrador.
Descripción: Comprobar las credenciales locales y guardar una sesión con identificación y rol.
Criterio de aceptación: El paciente navega al inicio y el administrador al panel después de un ingreso válido.

Condiciones de uso: Deben existir credenciales locales del paciente o la cuenta administrativa de demostración. La sesión debe poder guardarse en el navegador.

Desarrollo y resultado esperado: El sistema compara las credenciales y prepara los datos de sesión con su rol. Solo después de guardar la sesión debe actualizar el estado y continuar al destino correspondiente. La sesión del paciente contiene la información necesaria para la interfaz y no debe tratarse como una autorización validada por un servidor.

3.2.13 RF-13 Validar inicio de sesión

Actores: paciente y administrador.
Descripción: Exigir correo y contraseña y mostrar un error cuando las credenciales o el acceso no corresponden.
Criterio de aceptación: Un paciente no obtiene sesión desde el acceso administrativo; un fallo de guardado no navega.

Condiciones de uso: El usuario debe encontrarse en el acceso para pacientes o en el acceso administrativo, según la opción seleccionada.

Desarrollo y resultado esperado: Los campos vacíos o las credenciales incorrectas impiden completar el ingreso. El acceso administrativo debe rechazar una cuenta de paciente aunque sus credenciales sean válidas. Si no es posible persistir la sesión, la aplicación informa el fallo y no debe mostrar un ingreso exitoso ni navegar como si hubiera concluido.

3.2.14 RF-14 Procesar formulario de contacto

Actores: visitante y paciente.
Descripción: Validar nombre, correo y mensaje y mostrar un resultado de envío simulado.
Criterio de aceptación: El envío válido limpia los campos y aclara que no se envió un correo real.

Condiciones de uso: El formulario debe recibir nombre, correo y mensaje válidos según sus reglas.

Desarrollo y resultado esperado: El usuario envía el formulario y obtiene un resultado de la validación. En un envío válido se limpian los campos y se comunica expresamente el carácter simulado. El resultado no implica recepción por parte de la clínica, creación de un ticket ni despacho efectivo de un correo electrónico.

-

3.2.15 RF-15 Validar formulario de contacto

Actores: visitante y paciente.
Descripción: Rechazar campos vacíos, correo inválido y nombre o mensaje compuestos solo por espacios.
Criterio de aceptación: El formulario conserva los datos ante errores y señala los campos afectados.

Condiciones de uso: La validación se realiza sobre los valores introducidos antes de aceptar el envío.

Desarrollo y resultado esperado: Se comprueban obligatoriedad, longitud y formato, incluidos textos compuestos únicamente por espacios. El error debe permitir identificar el campo que requiere corrección. La información introducida se conserva durante la corrección para evitar que el usuario deba completar nuevamente todo el formulario.

3.2.16 RF-16 Mostrar validaciones y retroalimentación

Actores: todos los usuarios.
Descripción: Presentar mensajes comprensibles durante la interacción y el envío según cada formulario.
Criterio de aceptación: Editar un dato permite corregir su error y evita mantener mensajes generales de resultados anteriores.

Condiciones de uso: Los formularios y operaciones deben exponer al usuario el resultado correspondiente a su estado actual.

Desarrollo y resultado esperado: Los mensajes deben distinguir datos inválidos, operaciones exitosas y fallos de persistencia. Una corrección debe permitir actualizar la retroalimentación del formulario. No debe mantenerse un mensaje de éxito anterior de forma que sugiera que un contenido recién modificado ya fue procesado.

3.2.17 RF-17 Seleccionar nutricionista para una cita

Actores: visitante y paciente.
Descripción: Permitir seleccionar un profesional existente en el catálogo administrativo.
Criterio de aceptación: La reserva identifica al profesional y rechaza una referencia que no sea válida.

Condiciones de uso: El catálogo debe disponer de al menos un profesional válido para la consulta que se desea preparar.

Desarrollo y resultado esperado: El usuario elige al profesional y la reserva conserva su identificación. La validación debe impedir incorporar una referencia que no corresponda a un registro disponible. Esta selección identifica quién realizará la atención de demostración y no modifica la tarifa del plan seleccionado.

-

3.2.18 RF-18 Seleccionar fecha y hora

Actores: visitante y paciente.
Descripción: Permitir elegir una fecha futura y un horario del conjunto de atención configurado.
Criterio de aceptación: Las opciones de horario son 09:00, 11:00, 15:00 y 17:00.

Condiciones de uso: La persona debe elegir una fecha y uno de los horarios configurados en el formulario.

Desarrollo y resultado esperado: El sistema interpreta conjuntamente fecha y hora para determinar si la atención todavía es futura. Elegir la fecha de hoy no basta si la hora seleccionada ya pasó. Los horarios representan las opciones del prototipo y no una agenda clínica sincronizada con un servicio externo.

3.2.19 RF-19 Registrar motivo de consulta

Actores: visitante y paciente.
Descripción: Solicitar el motivo y conservar su texto normalizado en la reserva.
Criterio de aceptación: Solo se admite un motivo de entre 5 y 100 caracteres después de quitar espacios exteriores.

Condiciones de uso: El usuario debe proporcionar un motivo dentro del intervalo de longitud definido para agendamiento.

Desarrollo y resultado esperado: El texto se incorpora a la consulta después de retirar espacios exteriores. Una entrada vacía, demasiado corta o que supere el límite impide agregar la atención. El motivo sirve para describir la solicitud dentro de la simulación; no se utiliza para emitir un diagnóstico ni seleccionar automáticamente un tratamiento.

3.2.20 RF-20 Validar fecha y conflictos

Actores: visitante y paciente.
Descripción: Impedir fechas y horas pasadas y otra reserva del mismo profesional en la misma fecha y horario.
Criterio de aceptación: El conflicto se comprueba frente al carrito y las citas confirmadas y se informa sin agregar la consulta.

Condiciones de uso: La validación debe considerar la consulta propuesta, las reservas pendientes y las citas ya registradas localmente.

Desarrollo y resultado esperado: La combinación de profesional, fecha y hora identifica el conflicto que se debe rechazar. Ante un conflicto, se conserva el estado anterior y se informa al usuario para que elija otra alternativa. La regla opera sobre los datos de ese navegador y no garantiza exclusividad frente a reservas realizadas desde otros dispositivos.

-

3.2.21 RF-21 Agregar consulta al carrito

Actores: visitante y paciente.
Descripción: Guardar plan, profesional, motivo, fecha, hora y precio de una consulta válida.
Criterio de aceptación: Se actualizan el listado y el contador del Navbar y se muestra el resultado de la incorporación.

Condiciones de uso: El plan, profesional, motivo, fecha y hora deben superar las validaciones antes de modificar el carrito.

Desarrollo y resultado esperado: La incorporación genera una consulta pendiente con los datos necesarios para su revisión. La interfaz debe reflejar la cantidad actualizada y permitir comprobar lo agregado. Esta operación todavía no representa una cita confirmada: el usuario debe continuar al carrito y completar su revisión.

3.2.22 RF-22 Visualizar resumen del carrito

Actores: visitante y paciente.
Descripción: Mostrar las consultas pendientes con su servicio, profesional, fecha, hora, motivo y precio.
Criterio de aceptación: El carrito vacío ofrece accesos para agendar o consultar planes.

Condiciones de uso: El carrito puede contener una o varias consultas pendientes, o encontrarse vacío.

Desarrollo y resultado esperado: Cuando existen consultas, se muestran los datos necesarios para reconocer cada atención y revisar su importe. Si está vacío, se ofrece una forma de continuar hacia los servicios o el agendamiento. La representación del carrito debe corresponder al estado recuperado o modificado, sin presentar una confirmación que aún no ocurrió.

3.2.23 RF-23 Calcular total del carrito

Actores: visitante y paciente.
Descripción: Calcular el total a partir de los precios válidos de los planes de las reservas.
Criterio de aceptación: Agregar o eliminar una consulta actualiza el total; no se incorpora un recargo del profesional.

Condiciones de uso: Cada consulta debe corresponder a un plan válido del catálogo y a su precio aplicable.

Desarrollo y resultado esperado: El cálculo suma los importes de las consultas presentes y debe mantenerse coherente con el detalle visible. Una eliminación reduce el total por el valor de la atención retirada. Los valores se presentan en pesos chilenos y no se añaden cargos individuales de nutricionistas ni un descuento que no esté definido en el catálogo.

-

3.2.24 RF-24 Eliminar consulta del carrito

Actores: visitante y paciente.
Descripción: Permitir quitar una consulta antes de confirmar y actualizar la persistencia local.
Criterio de aceptación: La consulta deja de figurar y su valor se descuenta del total y del contador.

Condiciones de uso: Debe existir una consulta pendiente que el usuario decida retirar antes de confirmar.

Desarrollo y resultado esperado: La eliminación afecta exclusivamente a la consulta seleccionada y actualiza el listado, el contador y el total. El estado persistido debe reflejar la selección restante. Retirar una consulta del carrito no debe confundirse con cancelar una cita ya confirmada, porque corresponde a una etapa previa del flujo.

3.2.25 RF-25 Confirmar agendamiento

Actores: visitante y paciente.
Descripción: Revalidar las reservas y guardar las citas con datos del paciente antes de vaciar el carrito.
Criterio de aceptación: El éxito muestra identificadores y resumen confirmado; un fallo conserva el carrito y permite reintentar sin duplicar citas.

Condiciones de uso: El carrito debe contener reservas válidas y los datos de contacto de la revisión deben cumplir sus reglas.

Desarrollo y resultado esperado: La confirmación vuelve a comprobar las reservas, conserva el precio del plan y guarda las citas antes de vaciar la selección pendiente. Un resultado exitoso permite identificar las citas creadas. Si una operación falla, el usuario debe poder reintentar conservando sus consultas y sin generar nuevamente citas que ya quedaron registradas.

3.2.26 RF-26 Conservar citas y carrito

Actores: visitante y paciente.
Descripción: Recuperar los datos locales después de recargar o volver a montar el proveedor.
Criterio de aceptación: Las reservas válidas siguen disponibles y los datos ilegibles no se sustituyen silenciosamente por una lista vacía.

Condiciones de uso: La recuperación se realiza desde las claves de almacenamiento del mismo navegador y origen.

Desarrollo y resultado esperado: Al recargar, las consultas válidas deben seguir disponibles para continuar el flujo. Un contenido ilegible o un acceso bloqueado debe tratarse como una condición de error, no como autorización para sobrescribir los datos originales. La conservación se verifica también después de desmontar y volver a montar el proveedor del carrito.

-

3.2.27 RF-27 Acceder al panel administrador

Actores: administrador.
Descripción: Mostrar un dashboard con menú propio, resumen y accesos a la gestión.
Criterio de aceptación: Sin sesión se redirige a login; un paciente vuelve al inicio; solo rol admin visualiza el panel.

Condiciones de uso: Debe existir una sesión local cuyo rol sea administrador para mostrar el contenido de gestión.

Desarrollo y resultado esperado: La protección de rutas se aplica al acceso directo y a la navegación desde el sitio. El panel utiliza su propio layout para organizar dashboard, pacientes y nutricionistas. Las redirecciones de visitantes y pacientes evitan mostrar esa interfaz a roles no admitidos dentro de la simulación; no reemplazan controles de autorización del lado servidor.

3.2.28 RF-28 Gestionar pacientes

Actores: administrador.
Descripción: Crear, listar, editar y eliminar registros de pacientes mediante formularios y tablas.
Criterio de aceptación: Los cambios válidos se persisten y editar con contraseña vacía conserva la anterior.

Condiciones de uso: El administrador debe ingresar al módulo de pacientes y disponer de almacenamiento válido para la operación.

Desarrollo y resultado esperado: La creación añade un registro; la edición modifica el paciente seleccionado conservando su identidad y los datos que no deban reemplazarse. Una contraseña vacía durante la edición conserva la existente. La eliminación sigue las restricciones de integridad descritas en RF-39 y no debe ejecutarse como un efecto secundario de editar.

3.2.29 RF-29 Validar datos de pacientes

Actores: administrador.
Descripción: Comprobar RUN, correo, contacto, dirección y ubicación y evitar identificadores duplicados.
Criterio de aceptación: Los errores impiden guardar y se conserva la información anterior ante un fallo de almacenamiento.

Condiciones de uso: Los formularios administrativos deben aplicar las reglas de datos del paciente y contrastar identificadores con los registros existentes.

Desarrollo y resultado esperado: Un error de formato o duplicidad impide guardar el cambio y debe ofrecer información para corregirlo. En edición, la comprobación de duplicados debe distinguir el propio registro de otros pacientes. Los fallos de persistencia no deben presentar una actualización como completada ni eliminar silenciosamente la información anterior.

-

3.2.30 RF-30 Gestionar nutricionistas

Actores: administrador.
Descripción: Crear, listar, editar y eliminar nombre, especialidad, experiencia, modalidad y fotografía del catálogo.
Criterio de aceptación: La gestión funciona sin solicitar precio de consulta y se refleja en el sitio público.

Condiciones de uso: El administrador debe seleccionar creación, edición o eliminación dentro del módulo de nutricionistas.

Desarrollo y resultado esperado: El catálogo conserva los datos de identificación y presentación profesional que utiliza el sitio público. Al guardar un cambio válido, los componentes que consumen esos datos deben reflejarlo. El formulario no exige precioConsulta: la información profesional y las tarifas de los planes tienen responsabilidades distintas dentro del modelo.

3.2.31 RF-31 Aplicar los precios de los planes

Actores: visitante, paciente y administrador.
Descripción: Usar las tarifas de los servicios: consulta inicial 35000, plan integral 60000 y nutrición deportiva 50000 pesos chilenos.
Criterio de aceptación: Elegir otro nutricionista no altera el precio; la reserva y la confirmación conservan la tarifa del plan válido.

Condiciones de uso: La consulta debe identificar uno de los planes definidos en el catálogo de demostración.

Desarrollo y resultado esperado: La tarifa debe ser consistente desde la visualización del servicio hasta el carrito y la confirmación. Seleccionar otro profesional no modifica el importe. La oferta de primera consulta agrega el beneficio publicado sin reducir el precio base. La comprobación del importe debe apoyarse en el plan válido y no aceptar un precio arbitrario como autoridad del cálculo.

3.2.32 RF-32 Visualizar y buscar datos administrativos

Actores: administrador.
Descripción: Mostrar tablas, búsqueda y acciones de pacientes y nutricionistas.
Criterio de aceptación: La búsqueda filtra por los datos disponibles y el dashboard presenta pacientes, profesionales y citas del día.

Condiciones de uso: El administrador debe abrir el listado o dashboard correspondiente a la información que desea consultar.

Desarrollo y resultado esperado: La búsqueda reduce los registros visibles conforme al criterio disponible sin borrar los datos persistidos. El dashboard presenta cantidades calculadas a partir de los datos locales. El conteo de citas de hoy utiliza la fecha local y descarta las citas marcadas como canceladas; no representa una estadística centralizada de todos los dispositivos.

3.2.33 RF-33 Filtrar el catálogo por categoría

Actores: visitante y paciente.
Descripción: Presentar todos los servicios o filtrar evaluación inicial, atención integral o nutrición deportiva.
Criterio de aceptación: La categoría se refleja en la URL y el contador; una categoría desconocida muestra un aviso y permite restablecer el catálogo.

Condiciones de uso: El catálogo debe disponer de servicios con las categorías definidas para la aplicación.

Desarrollo y resultado esperado: El usuario puede limitar los resultados a una categoría y regresar al conjunto completo. La URL conserva el criterio seleccionado para relacionar navegación y contenido visible. Una categoría desconocida debe informarse de forma comprensible y ofrecer el restablecimiento del catálogo, evitando que el usuario interprete el resultado como una falla general.

3.2.34 RF-34 Consultar ofertas y beneficios

Actores: visitante y paciente.
Descripción: Mostrar la medición de composición corporal sin costo adicional en la primera evaluación y consulta inicial, con las condiciones publicadas.
Criterio de aceptación: Se indica el precio de 35000 CLP, primera consulta reservada por el sitio y medición presencial en clínica; el beneficio no reduce el precio del plan.

Condiciones de uso: La página de ofertas debe presentar el beneficio y las condiciones asociadas a la primera consulta.

Desarrollo y resultado esperado: El usuario debe poder reconocer qué incluye el beneficio, cuándo corresponde y que la medición se realiza presencialmente en clínica. El acceso para reservar debe conducir al plan de consulta inicial. El beneficio no constituye una rebaja monetaria automática ni cambia los precios de otros servicios.

3.2.35 RF-35 Preseleccionar un plan desde un enlace

Actores: visitante y paciente.
Descripción: Abrir agendamiento con el plan indicado en el parámetro plan de la URL.
Criterio de aceptación: Los accesos del catálogo y de la oferta seleccionan el servicio correspondiente.

Condiciones de uso: El acceso de origen debe incluir un identificador de plan reconocido por agendamiento.

Desarrollo y resultado esperado: Al abrir la página, el formulario selecciona el servicio indicado para conservar la intención de navegación. La preselección facilita continuar el proceso, pero no equivale a una consulta válida ni crea una cita por sí sola. El resto de los campos y las validaciones siguen siendo necesarios antes de agregar la reserva.

-

3.2.36 RF-36 Revisar los datos antes de confirmar

Actores: visitante y paciente.
Descripción: Abrir una revisión dentro de /carrito con resumen y campos de nombre y correo.
Criterio de aceptación: Volver al carrito conserva las consultas y no crea citas; con sesión se precargan los datos sin modificar la cuenta.

Condiciones de uso: Deben existir consultas pendientes para avanzar a la revisión dentro de la ruta del carrito.

Desarrollo y resultado esperado: La revisión reúne las consultas y solicita nombre y correo de contacto. Si existe sesión, los datos pueden precargarse y ajustarse únicamente para la reserva. Regresar al paso anterior debe conservar la selección y no escribir citas. Esta separación permite corregir la solicitud antes de ejecutar la confirmación.

3.2.37 RF-37 Simular un fallo de confirmación

Actores: visitante y paciente.
Descripción: Ofrecer un escenario de fallo para comprobar la conservación de reservas y la posibilidad de reintento.
Criterio de aceptación: El resultado informa el error y conserva las consultas; al reintentar con éxito no se duplican las citas.

Condiciones de uso: El escenario de fallo debe seleccionarse desde la revisión cuando el usuario desea comprobar el comportamiento ante una confirmación fallida.

Desarrollo y resultado esperado: La simulación permite observar el mensaje de error y la conservación del carrito. Al retirar la condición de fallo y volver a confirmar, el flujo debe poder completarse sin duplicar registros. El mecanismo es una ayuda de demostración y prueba; no representa el rechazo de una tarjeta ni una transacción bancaria.

3.2.38 RF-38 Cerrar sesión y adaptar el menú

Actores: paciente y administrador.
Descripción: Permitir cerrar sesión, retirar la sesión local y actualizar las opciones visibles.
Criterio de aceptación: El menú de visitante muestra login y registro y el enlace Administración se ofrece al rol admin.

Condiciones de uso: Debe existir una sesión de paciente o administrador para ejecutar el cierre.

Desarrollo y resultado esperado: La operación retira el estado de sesión y solicita eliminar su persistencia. La navegación posterior debe reflejar las opciones de visitante. Cerrar sesión no debe confundirse con borrar los pacientes, los profesionales o las citas; esos datos tienen claves y responsabilidades propias. La comprobación incluye la actualización de los enlaces visibles del menú.

-

3.2.39 RF-39 Conservar integridad al eliminar registros

Actores: administrador.
Descripción: Solicitar confirmación y bloquear la eliminación de un paciente o profesional referenciado en citas o reservas.
Criterio de aceptación: Cancelar no elimina; una referencia existente muestra una advertencia y conserva el registro.

Condiciones de uso: El administrador debe seleccionar un registro y confirmar expresamente la intención de eliminarlo.

Desarrollo y resultado esperado: Antes de completar la eliminación se comprueban las referencias consideradas por la aplicación. Si existen citas o reservas asociadas, el registro se conserva y se informa el impedimento. Si el usuario cancela la confirmación, no debe haber cambios. La regla busca evitar referencias inconsistentes dentro del conjunto de datos locales.

3.2.40 RF-40 Resolver rutas y desplazamiento

Actores: todos los usuarios.
Descripción: Mostrar una página no encontrada para rutas desconocidas y desplazar al inicio o sección solicitada.
Criterio de aceptación: Volver al inicio elimina el desplazamiento anterior; Contacto llega a su sección; la recarga de rutas públicas funciona con Nginx.

Condiciones de uso: La aplicación debe recibir una ruta o una referencia a una sección y resolverla según la navegación configurada.

Desarrollo y resultado esperado: Las rutas desconocidas deben ofrecer una salida hacia el inicio. El desplazamiento debe ajustarse al destino para evitar que una nueva página se abra en una posición heredada que oculte su encabezado. En producción, la recarga requiere que Nginx entregue index.html y permita a React Router interpretar la dirección solicitada.

3.3 Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad, restricciones técnicas y condiciones generales que debe cumplir el sistema NutriVida.

3.3.1 Requisitos de rendimiento

La aplicación debe responder a interacciones y cambios de estado durante su uso normal. No se establece un SLA, un tiempo numérico máximo de respuesta ni capacidad de usuarios simultáneos, porque la EV2 es un prototipo frontend.

El total y contador del carrito, los filtros y mensajes se actualizan al cambiar el estado. La confirmación bloquea envíos repetidos mientras se procesa la operación.

La revisión del rendimiento se concentra en la respuesta observable durante los recorridos de demostración: cambiar filtros, abrir detalles, corregir campos y actualizar reservas. El resultado satisfactorio exige que el estado visible corresponda a la acción realizada y que una operación pendiente no permita confirmar repetidamente la misma solicitud. No se infieren capacidades de carga a partir de los tiempos de las pruebas unitarias.

3.3.2 Seguridad

RNF-04 Proteger la integridad de los datos

El sistema debe rechazar datos inválidos y comprobar el rol para mostrar las rutas administrativas. El registro público siempre asigna el rol paciente. Los fallos de almacenamiento deben impedir declarar éxito sin guardar la operación.

La sesión y las credenciales locales son una simulación; no constituyen autenticación segura de producción ni protección para información clínica real. En las demostraciones se utilizan datos ficticios. La seguridad del servidor se reserva para una evolución con backend.

El control de roles organiza la experiencia de usuario, pero una persona con acceso a las herramientas del navegador puede inspeccionar o modificar el almacenamiento local. Por este motivo, la información utilizada en la entrega debe ser de demostración. La aplicación no debe presentarse como una plataforma apta para custodiar historias clínicas ni como un sistema con autorización verificable desde un backend.

3.3.3 Fiabilidad

RNF-07 Mantener la persistencia local

El sistema debe conservar pacientes, profesionales, sesión, carrito y citas después de recargar en el mismo origen. Los fallos de lectura o escritura deben mostrarse y evitar pérdidas silenciosas; los datos ilegibles no deben sobrescribirse como una lista inicial vacía.

La persistencia depende del navegador, dispositivo y origen. No se garantiza sincronización entre dispositivos o pestañas concurrentes. El reintento de una confirmación debe evitar citas duplicadas.

La fiabilidad de una operación se evalúa mediante su resultado completo: validar los datos, intentar guardarlos, actualizar el estado que corresponda y comunicar el resultado. Si una etapa falla, la aplicación debe evitar anunciar un éxito inexistente. En el carrito se presta especial atención al orden de guardado de citas y limpieza de reservas, porque un reintento no debe producir pérdida de la selección ni duplicación de confirmaciones.

3.3.4 Disponibilidad

El sistema se publica en AWS EC2 con Ubuntu Server 24.04 LTS y Nginx. El servicio continúa funcionando al cerrar SSH mientras la instancia y el laboratorio estén activos.

La disponibilidad depende de AWS Academy y su presupuesto; no se garantiza un porcentaje. La IPv4 pública registrada al verificar el despliegue es 54.160.216.249 y puede cambiar al detener e iniciar la instancia. Las rutas internas deben admitir recarga.

3.3.5 Mantenibilidad

RNF-06 Mantener el código organizado

El proyecto debe separar componentes, páginas, contextos, datos, utilidades y estilos. Los componentes reutilizables deben recibir propiedades y compartir estado mediante contextos cuando corresponda. Vite gestiona el desarrollo y la compilación.

El mantenimiento del proyecto será realizado por los integrantes del equipo mediante ramas, commits y revisiones en GitHub.

Los cambios en reglas compartidas deben revisarse en todos sus consumidores. Por ejemplo, modificar una tarifa requiere comprobar catálogo, agendamiento, carrito y confirmación; modificar un formato de paciente requiere comprobar registro, administración e inicio de sesión. La trazabilidad y las pruebas permiten identificar esas relaciones antes de integrar una modificación en la rama principal.

3.3.6 Portabilidad

RNF-01 Garantizar diseño responsivo

La revisión responsive debe cubrir 360, 768 y 1280 px y anchos intermedios. Se verifican sitio público, catálogo, ofertas, carrito, formularios y panel administrativo, sin pérdida de contenido.

RNF-05 Garantizar compatibilidad web

El sistema debe poder utilizarse en navegadores modernos con JavaScript y localStorage. No se declara una certificación de compatibilidad para navegadores que no se hayan verificado.

El sistema podrá ejecutarse en computadores, tabletas y teléfonos móviles sin requerir la instalación de programas adicionales.

-

3.3.7 Usabilidad y accesibilidad

RNF-02 Mantener una interfaz consistente

El sistema debe utilizar colores, tipografías, botones, barra de navegación, formularios y pie de página coherentes en todas sus páginas.

RNF-03 Garantizar accesibilidad básica

El sistema debe utilizar textos alternativos, contraste legible, etiquetas asociadas a campos y nombres accesibles en controles. Los estados de expansión y errores deben poder identificarse cuando corresponda. Se exige accesibilidad básica, sin declarar una certificación WCAG.

RNF-08 Proporcionar retroalimentación al usuario

El sistema debe mostrar errores, éxito y confirmaciones comprensibles. Un fallo de confirmación conserva el carrito y permite reintentar; contacto aclara el envío simulado. Las modificaciones permiten corregir los errores sin perder datos.

La revisión de usabilidad considera que una persona pueda identificar dónde se encuentra, qué información se solicita y qué ocurrió después de su acción. La accesibilidad básica se verifica mediante etiquetas, nombres de controles y estados expuestos en la interfaz, complementada con inspección visual. Las pruebas en jsdom aportan evidencia de interacción, pero no sustituyen la comprobación del diseño en un navegador real.

3.4 Otros Requisitos

Los requisitos adicionales de esta entrega abarcan el proceso de pruebas unitarias, la documentación de resultados y la publicación de producción. Las siguientes reglas y evidencias complementan los requisitos funcionales y no funcionales.

3.4.1 Reglas de validación

Registro: nombre entre 2 y 50 caracteres y apellidos entre 2 y 100, solo letras y espacios entre palabras; RUN con dígito verificador válido; correo de hasta 100 caracteres de los dominios duoc.cl, profesor.duoc.cl o gmail.com. RUN y correo no deben repetirse.

Datos del paciente: teléfono 912345678 o +56912345678; región y comuna del catálogo disponible; dirección de 5 a 300 caracteres; fecha de nacimiento opcional, válida, desde 1900 y no futura. Contraseña de al menos seis caracteres, no solo espacios, y confirmación coincidente en el registro.

Contacto: nombre de 2 a 100 caracteres, correo válido de hasta 100 y mensaje de 2 a 500; se rechaza texto formado solo por espacios. Su resultado no genera un correo real.

-

Nutricionista: nombre profesional de 3 a 100 caracteres, especialidad de 3 a 150, experiencia de 10 a 500, modalidad y fotografía del catálogo. No se solicita precioConsulta.

Agendamiento: plan y profesional existentes, motivo de 5 a 100 caracteres, fecha y hora futuras y horarios 09:00, 11:00, 15:00 y 17:00. Se impide el conflicto por profesional, fecha y hora frente a citas y reservas.

Revisión del carrito: nombre de 2 a 100 caracteres y correo válido de hasta 100. El correo de contacto de la reserva no se restringe a los dominios del registro. Estos datos se guardan en la cita y no modifican la cuenta.

Edición y eliminación: una contraseña vacía al editar conserva la anterior. La eliminación requiere confirmación y se bloquea cuando el registro tiene citas o reservas asociadas.

Las reglas se aplican al propósito de cada formulario. Los dominios admitidos para registrar una cuenta no se trasladan automáticamente al correo de contacto de una reserva. La validación del carrito permite utilizar un correo con formato válido dentro de su límite de longitud, porque ese dato identifica a quién corresponde la cita y no crea por sí mismo una cuenta registrada.

La normalización debe conservar el significado de los datos. Quitar separadores del RUN o convertir el correo a minúsculas facilita las comparaciones; no justifica eliminar registros anteriores ni aceptar una fecha inexistente. Ante un error, los mensajes deben describir qué necesita corregirse y mantener la información que permita continuar el formulario.

3.4.2 RNF-09 Proceso de pruebas y cobertura

La aplicación debe configurar Vitest, Testing Library, jsdom y coverage-v8 para verificar funciones e interacciones del frontend. Las pruebas incluyen casos positivos y negativos, cambios de estado, navegación, validaciones, CRUD, persistencia y recuperación de errores.

Los mocks vi.fn y vi.spyOn controlan respuestas y fallos del almacenamiento. Las pruebas de fechas utilizan un reloj controlado y restauran sus temporizadores. Cada caso prepara sus datos y limpia los componentes y mocks según el entorno.

Desde fase2/proyecto_etapa2_react se ejecutan npm run test -- --run y npm run test:coverage. La medición incluye JavaScript y JSX de src y excluye archivos de pruebas, setupTests.js y main.jsx. Genera reporte HTML, resumen en terminal y coverage-summary.json.

La ejecución de referencia del 16/10/2026 a las 02:20:55, hora de Chile, aprobó 151 pruebas en 15 archivos en 34,07 segundos. Cobertura: sentencias 90,29 % (791/876), ramas 81,89 % (638/779), funciones 93,44 % (228/244) y líneas 90,20 % (764/847). Son resultados documentados; no se declara una nueva ejecución para esta ERS.

El informe de pruebas y el reporte se conservan en docs/ev2/pruebas_unitarias/. La cobertura mide código ejecutado y no garantiza ausencia de defectos. La rúbrica requiere explicar diez pruebas representativas, no diez archivos ni un porcentaje numérico específico.

-

El análisis de cobertura debe interpretar cada métrica por separado. Las sentencias y líneas indican cuánto código se ejecutó; las funciones reflejan cuántas unidades invocables participaron; las ramas muestran qué alternativas de decisión fueron recorridas. Una cobertura alta aporta información sobre el alcance de las pruebas, pero requiere aserciones útiles para demostrar resultados correctos.

Los casos positivos comprueban operaciones válidas y sus efectos; los negativos verifican rechazos y conservación del estado. Los mocks permiten reproducir fallos de lectura o escritura que no aparecen necesariamente durante una demostración normal. La restauración de mocks y temporizadores evita que un caso altere las condiciones del siguiente y favorece resultados reproducibles.

Cuando se modifica el comportamiento de una función o componente, se deben revisar los casos relacionados y regenerar la evidencia si corresponde a una nueva versión de la aplicación. El reporte HTML, el resumen de cobertura y las capturas deben identificar resultados de una misma ejecución de referencia. Una actualización editorial de la ERS no equivale a ejecutar nuevamente las pruebas.

3.4.3 RNF-10 Publicación en AWS EC2

La compilación npm run build genera dist, que Nginx publica en /var/www/html. La configuración de rutas utiliza try_files $uri $uri/ /index.html. Debe permitir abrir y recargar rutas como /registro, /servicios, /ofertas y /carrito.

Se verifica sudo nginx -t con syntax is ok y test is successful, la recarga del servicio y sudo systemctl is-active nginx con resultado active. La publicación registrada es http://54.160.216.249 y utiliza HTTP, sin certificado HTTPS.

El procedimiento y las capturas están en docs/ev2/despliegue_aws/. Si la IP pública cambia, se actualizan los enlaces y evidencias. La clave privada permanece fuera del repositorio.

La actualización del sitio publicado debe partir de los cambios integrados en main. Se instalan las dependencias registradas, se genera una compilación correcta y posteriormente se copian sus recursos al directorio servido por Nginx. El paso de publicación debe realizarse después de verificar la compilación para evitar presentar como actualizada una versión que no pudo generarse.

La verificación del despliegue combina comprobación técnica del servicio y navegación desde la dirección pública. Se debe abrir una ruta interna directamente y recargarla, además de acceder desde los enlaces del sitio. Este recorrido distingue una ruta resuelta por React de un error 404 emitido por el servidor. Las capturas deben permitir reconocer la dirección utilizada y el contenido que se revisó.

3.4.4 Trazabilidad y criterios de entrega

RF-01 a RF-07: Inicio, Nosotros, listado de profesionales y artículos; App.test.jsx y ListaNutricionistas.test.jsx, complementados con revisión visual del contenido y video.

RF-08 a RF-13 y RF-38: Registro, Login, AuthContext y Navbar; Registro.test.jsx, Login.test.jsx, AuthContext.test.jsx y Navbar.test.jsx.

-

RF-14 a RF-16: FormularioContacto y formularios validados; FormularioContacto.test.jsx y pruebas de interacción de las páginas.

RF-17 a RF-26, RF-31, RF-35 a RF-37: Agendar, Carrito y CarritoContext; Agendar.test.jsx, Carrito.test.jsx y CarritoContext.test.jsx.

RF-27 a RF-30, RF-32 y RF-39: RutaAdmin, AdminLayout, AdminListado, AdminFormulario y DatosContext; Admin.test.jsx y App.test.jsx.

RF-33 y RF-34: Servicios, Ofertas y PlanesServicios; Catalogo.test.jsx y PlanesServicios.test.jsx. RF-40: App y ScrollNavegacion; App.test.jsx y ScrollNavegacion.test.jsx. Las utilidades se verifican en utils.test.js.

RNF-01 a RNF-10: revisión responsive en 360, 768 y 1280 px, pruebas de interfaz y contexto, informe de cobertura y evidencias de AWS. Las revisiones visuales complementan las pruebas en jsdom.

La entrega incluye enlace público del repositorio, proyecto frontend comprimido, ERS versión 2 e informe de cobertura. La presentación individual explica React, props, estado, Bootstrap, diez pruebas representativas, mocks y análisis de resultados.

La trazabilidad expresa una relación entre requisitos, módulos y evidencia; no significa que un único archivo de prueba cubra exhaustivamente todos los comportamientos de un grupo. Las comprobaciones automatizadas se complementan con la revisión responsive, la inspección del contenido y la validación de rutas publicadas. Cada conclusión debe apoyarse en el tipo de evidencia adecuado.

La revisión final debe comprobar coherencia entre esta ERS, los README, el informe de pruebas y la aplicación entregada. Los enlaces deben conducir a las ubicaciones documentadas y las cifras de pruebas deben corresponder al reporte de referencia. La presentación permite explicar las decisiones de implementación y demostrar casos representativos, incluidos errores, cambios de estado y conservación de datos.